

PseudoDOM

A Client-Side DOM Rewriting Architecture for Safe CSR Reconciliation

Architecture Specification

Part One: the original concept, as proposed · Part Two: framework-specific refinement

Prepared for architecture-diagram finalization — 2026-08-04

Table of Contents

Table of Contents	2
0. Purpose and Motivation	4
1. Background: Why This Problem Exists	4
1.1 Browsers Never Reject Malformed HTML	4
1.2 Foster Parenting and the Adoption Agency Algorithm	4
1.3 The One Fact This Entire Architecture Rests On	5
1.4 mXSS in One Sentence	5
PART ONE — The Original Concept (PseudoDOM, as Proposed)	6
2.1 Core Premise	6
2.2 The Staging Area	6
2.3 Event Handler Tracking	6
2.4 Sanitization Strategy	6
2.5 Performance: Operation Tracking and Batching.....	7
2.6 Positioning Relative to Trusted Types.....	7
PART TWO — Refined Architecture (Framework-Specific Suggestions).....	8
3.1 Two-Tier Interception Model	8
3.2 The Seven-Stage Pipeline	8
Stage 1 — Sink Interception.....	8
Stage 2 — Redirect to a Detached Parse Target.....	8
Stage 3 — Pre-Purification Inventory	9
Stage 4 — In-Place Purification.....	9
Stage 5 — Structural Fidelity Check	9
Stage 6 — Operation Ledger (Batch Before Sanitize, Not After)	9
Stage 7 — Promotion.....	10
3.3 Framework-Level Interception: Host Config / Custom Renderer Hooking	11
3.4 The Coverage Gap: Attribute and URL-Value Injection.....	11
Recommended second lane.....	12
3.5 Reconciliation Detail: Updates to Already-Live Nodes	12
3.6 Trusted Types: Relationship, Not Competition.....	13
Where they genuinely differ	13
Recommendation.....	13
3.7 Native Platform Alternative Worth Tracking: the Sanitizer API.....	13
4. Consolidated Comparison Table	15

5. Threat Coverage Matrix	16
6. Data Flow Specification for the Architecture Diagram	17
6.1 Entry Points (top of diagram).....	17
6.2 Tier 1 Path (left branch)	17
6.3 Tier 2 Path (right branch, the main pipeline).....	17
6.4 Shared Merge Point	17
6.5 Two Optional Perimeter Integrations (side inputs, not part of the main flow).....	17
7. Summary and Final Recommendation	19
7.1 Validation Steps Before Committing to the Diagram as Final.....	19
Appendix A — Tier 2 (String-Sink) Surfaces to Intercept	20
Appendix B — Dangerous Attribute Names for the Tier 1 Lane (§3.4)	20

0. Purpose and Motivation

Client-side-rendered (CSR) applications write to the DOM through IDL calls issued after reconciliation — the diff-and-commit step that decides which nodes to create, update, move, or remove. Reconciliation itself happens entirely in memory; nothing touches the real document until the commit phase actually calls the DOM API.

The overwhelming majority of those calls are safe by construction. A minority — the explicit "raw HTML" escape hatches every framework exposes — are not, because they hand a string to the HTML parser, and the HTML parser is specified to recover from malformed input by rewriting the tree rather than rejecting it. If that recovery is not accounted for, sanitization performed against the wrong representation (the source string, or an assumed tree shape) can be silently bypassed once the real parser produces a different tree — the mutation XSS (mXSS) problem.

This document specifies PseudoDOM: an architecture that intercepts DOM-mutating IDL calls, stages any string that would trigger a parse in a detached construction area, purifies and structurally verifies the result before it ever reaches the live document, and batches the verified operations for performance.

The document is split into two parts, matching how the design developed. Part One documents the architecture exactly as originally proposed: wrapper interception, a template-based staging area, event-handler tracking, staged-tree sanitization, and operation batching, positioned as an opt-in, purely client-side alternative to Trusted Types. Part Two documents the refinements developed against a specific deployment target — CSR reconciler frameworks such as React and Vue — including a stronger interception point, a coverage gap the original design does not close, and how the two approaches (Trusted Types and PseudoDOM) are best combined rather than chosen between.

1. Background: Why This Problem Exists

A short grounding in the mechanics the rest of this document assumes, for anyone building the architecture diagram without the full context of the prior discussion.

1.1 Browsers Never Reject Malformed HTML

There is no such thing as a fatal HTML parse error in a browser. Every malformed-input condition the specification recognizes has a named, deterministic recovery action attached to it — the parser always produces a tree, and since the HTML5 tree-construction algorithm was standardized, every conformant browser produces the same tree for the same malformed input. "Malformed" and "renders successfully" are not opposites in HTML the way they are in most other formats.

1.2 Foster Parenting and the Adoption Agency Algorithm

Two of the named recovery mechanisms matter most for this design:

- **Foster parenting** — when content is inserted while the parser's current position is inside a table-family element that cannot host it, the content is relocated to a computed position outside the table instead of being dropped or nested somewhere invalid.

- **The adoption agency algorithm** — the repair procedure for formatting elements whose start and end tags straddle a structural boundary (e.g. a `<b>` left open across a `<div>`). Its relocation step can itself trigger foster parenting, so the two mechanisms compose.

Both are correct, specified, deterministic behavior — not bugs. The risk they create is entirely downstream: a check performed against one representation of the markup (the source string, or an assumed tree shape) can miss content that a correct parse relocates somewhere else.

1.3 The One Fact This Entire Architecture Rests On

Tree construction (parsing) and DOM graph mutation are different code paths.

Calls like `appendChild`, `insertBefore`, and `moveBefore` splice already-existing Node objects into the tree directly. They never invoke the tokenizer or the tree-construction algorithm, so foster parenting, the adoption agency algorithm, and every other parser-recovery mechanism are structurally incapable of firing through them — regardless of how deeply nested, how repeatedly staged, or how the nodes were originally created. Only APIs that hand the browser a raw string to interpret as markup (`innerHTML`/`outerHTML`, `insertAdjacentHTML`, `document.write`, `Range.createContextualFragment`, `DOMParser`) invoke the parser. This single fact is what makes IDL-level interception a tractable strategy: the risk surface is not “every DOM call,” it is a small, enumerable set of string-consuming sinks.

This distinction is the load-bearing assumption for both Part One and Part Two of this specification.

1.4 mXSS in One Sentence

A defense that sanitizes one representation of markup (a string, or a first parse) and then allows a second, independent parse to happen later — whether by a sanitizer’s own round-trip through a string, or by the eventual consumer of that string — is only as safe as the guarantee that both parses produce an identical tree. Where they can diverge, sanitization performed on the first representation can be silently bypassed by the second.

PART ONE — The Original Concept (PseudoDOM, as Proposed)

This part documents the architecture exactly as originally proposed, before framework-specific refinement. It is the baseline Part Two builds on.

2.1 Core Premise

Existing DOM mutation APIs are overridden with wrappers that act as a single, common interception junction for every call that would otherwise reach the live DOM directly. Rather than allowing application code to write to the document immediately, every intercepted call is routed through this junction first, which is where the rest of the architecture attaches.

The stated goals for this junction, as proposed:

- Aggregate every DOM mutation call passing through it, rather than acting on each one independently.
- Build the DOM in an independent staging area before anything is attached to the live (“light”) DOM.
- Use the wrapper layer itself as the mechanism for identifying which calls are genuinely DOM operations occurring on the nodes being created.

2.2 The Staging Area

The staging area is an independent template element — a construction site the real document tree is never exposed to until the architecture decides it is safe to attach. Nodes are created in this area without attachment to the live document. Once a region of the staged tree has been checked, it is marked; only marked, verified regions are eligible to move from the staging area into the light DOM.

This reframes “attach to the DOM” from a single, immediate action into a two-phase one: construct off-document, then promote — giving the architecture a checkpoint between the two phases that a direct-attachment model does not have.

2.3 Event Handler Tracking

Every event handler encountered while building the staged tree is tracked, not just acted on. The proposal explicitly includes both categories — handlers that are benign and handlers that carry bad intent — checked against a whitelist, so that the architecture has a complete record of what handler surface exists in the staged content before any of it is promoted.

2.4 Sanitization Strategy

Three layers, all operating on the staging tree rather than on the original source string in isolation:

- Sanitization in specified regions of the staged tree, using DOMPurify.
- Escaping and output encoding as additional, complementary layers.
- An association step: textual components originating from string-based calls (the innerHTML-class case named explicitly) are tied back to their corresponding rewritten DOM nodes only after

the source strings have been purified — the purification happens before the association, not after.

2.5 Performance: Operation Tracking and Batching

Every intercepted operation is tracked rather than executed immediately, specifically to avoid the performance cost of issuing many discrete calls against the live DOM. Tracked operations are aggregated and applied together, rather than one at a time as application code originally issued them.

2.6 Positioning Relative to Trusted Types

The proposal is explicitly framed as distinct from the browser’s Trusted Types mechanism, on three stated grounds:

- Opt-in at the application level — a developer must explicitly integrate it for it to take effect.
- Purely client-side — does not depend on server-side configuration (such as a CSP header) to function.
- Usable by “even a novice browser user” — i.e., it is meant to function as a self-contained client library with no deployment-level prerequisites, in contrast to Trusted Types’ CSP-enforced mode.

PART TWO — Refined Architecture (Framework-Specific Suggestions)

This part documents the refinements developed once the deployment target was named explicitly: CSR applications built on reconciler frameworks (React, Vue, and structurally similar systems), where the motivating risk is DOM IDL calls issued after reconciliation without sanitization at that boundary.

3.1 Two-Tier Interception Model

The first refinement is a formal split of every interceptable call into two tiers, because they carry fundamentally different risk and need fundamentally different handling.

Tier	Representative calls	Parses?	Risk class	Handling
Tier 1 — Graph mutation	appendChild, insertBefore, moveBefore, replaceChild, removeChild, setAttribute (non-URL), textContent, node.data	No	None for tree shape (see §3.4 for attributes)	Batch for performance only
Tier 2 — String sink	innerHTML / outerHTML (set), insertAdjacentHTML, document.write / writeln, Range.createContextualFragment, DOMParser.parseFromString	Yes — full tree construction	Foster parenting, adoption agency, mXSS drift	Full seven-stage pipeline (§3.2)

This split resolves an open question from the original proposal directly: the wrapper layer does not need to infer, by trial and error, "which calls are actually DOM operations that matter." The call name alone determines the tier, and the tier alone determines whether the pipeline in §3.2 runs at all.

3.2 The Seven-Stage Pipeline

Full detail for every Tier 2 call. This is the part of the architecture the diagram's main flow should be built around.

Stage 1 — Sink Interception

The wrapper layer identifies a Tier 2 call as it happens — e.g. an assignment to `el.innerHTML` — and prevents the raw string from reaching the native setter. Implementation note: for the `innerHTML/outerHTML` case this requires overriding the property's accessor descriptor on the relevant prototype (it is a getter/setter pair, not a plain method), not a simple function reassignment.

Stage 2 — Redirect to a Detached Parse Target

The intercepted string is handed to a private, detached template element's content — never to the real target node directly. This is a full parse: the browser's tokenizer and tree-construction algorithm run exactly as they would anywhere else, including any foster parenting or adoption-agency-style recovery the string's shape requires. The critical property of this stage is not that it prevents that recovery from happening — it cannot — it is that the recovery happens in isolation, before anything is visible in the live document.

Whether the eventual target node is already attached to the live document or not yet attached is irrelevant at this stage — the string is redirected either way, and the real target is not touched until Stage 7.

Stage 3 — Pre-Purification Inventory

Before any content is removed or altered, the detached fragment is walked once to build a complete inventory. Two categories of event handler must be recorded separately, because they carry different trust levels by construction:

- **String-sourced handlers** — `on*` attributes, `javascript:` URLs, or similar found inside the parsed string content itself. These arrived embedded in untrusted text, not written by application code. The correct default is that none of these survive promotion, unconditionally — a whitelist of permitted values in this category is rarely the right model.
- **Application-sourced handlers** — listeners attached via `addEventListener` by trusted application code, already visible to the Tier 1 wrapper with full call-site provenance. A whitelist concept is meaningful here — it is whitelisting origins, not values.

Treating both categories as one undifferentiated "event handler" list, as in the original proposal, makes it impossible to apply the correct default to each — splitting by provenance at this stage is what resolves that.

Stage 4 — In-Place Purification

DOMPurify (or an equivalent) runs directly against the already-parsed, still-detached fragment, in place — not against the original string a second time. This distinction matters concretely: DOMPurify's default mode takes a string, parses it into its own internal document, sanitizes that internal tree, and serializes the result back out to a string, which would then require a second, independent parse to become real nodes again — a redundant extra parse boundary, and exactly the kind of hop the mXSS problem in §1.4 is about. Sanitizing the fragment produced in Stage 2 directly (DOMPurify's in-place node mode, or an equivalent DOM-native mode) collapses this to one parse, one tree, one thing to reason about.

Stage 5 — Structural Fidelity Check

A second walk of the purified fragment, checking that the actual parent/ancestor chain of each node matches what the calling code's intent implies — e.g., content meant to populate a specific container's children is still directly under that container, and nothing relocated somewhere unexpected during the Stage 2 parse. This is the concrete form of "marking a region safe": a region passes only if it is both content-clean (Stage 4) and shape-as-expected (this stage). A region can fail this check even after passing sanitization, if something structural happened that the calling code did not anticipate — that is itself worth surfacing, independent of exploitability.

Stage 6 — Operation Ledger (Batch Before Sanitize, Not After)

Each Tier 2 call produces a ledger entry — target node, purified fragment, call site — appended to a queue rather than flushed immediately. The ordering here is the single most important design decision in the pipeline: purification and the structural check (Stages 4–5) must run once, after every pending operation for a flush cycle has been aggregated — not once per individual call before batching. Sanitizing each call's fragment independently and only batching the already-sanitized pieces reopens a known bypass class: content that looks inert when checked in isolated fragments can combine into something that is not inert once assembled and reparsed together — the same fusion-across-boundaries

pattern demonstrated by table-context whitespace and text-node fusion in HTML tree construction generally. Aggregate first, purify the fully assembled result last, exactly once per flush.

Stage 7 — Promotion

The ledger is flushed using Tier 1 operations only — `moveBefore` / `appendChild` of the purified fragment's children into their real targets. This step is parse-free by construction (see §1.3), so nothing new can happen here that Stages 2–6 did not already account for.

```
Tier 2 call (innerHTML=, insertAdjacentHTML, document.write, ...)
  |
  v
[1] Sink Interception -----
  | (accessor override catches the raw string)
  v
[2] Redirect to Detached <template>.content -----
  | (full parse happens here, in isolation; foster
  |   parenting / adoption agency may fire, safely)
  v
[3] Pre-Purification Inventory -----
  | split: string-sourced handlers vs app-sourced handlers
  v
[4] In-Place Purification (DOMPurify, IN_PLACE) -----
  | no round-trip through a second string
  v
[5] Structural Fidelity Check -----
  | parent/ancestor chain matches caller intent?
  v
[6] Operation Ledger { target, purifiedFragment, callSite }
  | MANY calls aggregate here across a flush cycle
  | (purify/verify already happened per-entry in 4-5;
  |   the ledger itself does not re-purify on assembly –
  |   see note below on flush-cycle granularity)
  v
[7] Promotion -- Tier 1 ops only (appendChild / moveBefore)
  |
  v
Live (light) DOM
```

Figure 1. Tier 2 pipeline, single call. See Figure 2 for the batching/ordering detail across multiple calls in one flush cycle.

WRONG ORDER (reopens the fusion bypass):

```
call A --> parse --> purify A --> \
call B --> parse --> purify B --> }--> batch --> promote
call C --> parse --> purify C --> /
```

Each fragment looks clean alone; assembled + reparsed together downstream, content can fuse across the boundaries between them.

CORRECT ORDER (Stage 6 as specified):

```
call A --> parse (Stage 2) --> \
call B --> parse (Stage 2) --> }--> aggregate FIRST
```

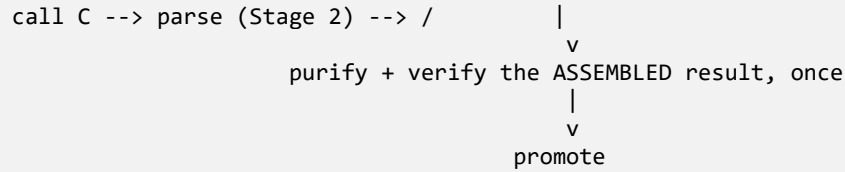


Figure 2. Why purification must run on the aggregated flush, not per individual call, before batching.

3.3 Framework-Level Interception: Host Config / Custom Renderer Hooking

This is the refinement specific to naming CSR reconciliation as the deployment target, and it changes the recommended interception strategy from §2.1.

Global prototype patching (overriding `Node.prototype.appendChild`, the `innerHTML` accessor on `Element.prototype`, etc.) has three structural weaknesses:

- Bypass via early reference — any code holding a reference to the native accessor captured before the patch was installed (a third-party script, a bundler that cached the descriptor early) calls the real DOM directly, invisibly to the wrapper.
- Iframe realm gaps — each frame has its own `Node.prototype` / `Element.prototype`; a global patch in the top-level document does not extend into an iframe's realm without separate injection.
- Operation-type inference — a patched prototype method does not itself say what kind of operation is occurring; the wrapper has to infer it from the call shape.

Reconciler frameworks already solve a version of this problem for their own purposes. React's reconciler funnels every DOM operation through a host config — an internal renderer backend with explicitly named operations (`createInstance`, `appendChild`, `commitUpdate`, and so on). Vue's custom-renderer API is structurally the same idea. These are first-class, intended extension points, not monkeypatches.

For a known-framework deployment, hooking at this layer instead of (or in addition to) global IDL patching resolves all three weaknesses at once for any code executing inside the framework's own render path: there is no "early reference" to bypass, because the framework always calls its own host config by construction; the framework's execution model is realm-agnostic in the way that matters here; and the host config already names the operation type directly, which is exactly the "which calls are actually DOM operations" distinction the original proposal was trying to reconstruct via wrapper inference in §2.1.

Recommendation: for a React/Vue-style target, treat global prototype patching as a fallback for code outside the framework's control (third-party widgets, non-framework legacy code), and treat the host-config / custom-renderer hook as the primary interception point for everything the framework itself renders.

3.4 The Coverage Gap: Attribute and URL-Value Injection

Neither §2 nor §3.1–3.3 covers this class, and it needs to be closed separately rather than folded into the Tier 2 pipeline.

An ordinary reconciler-driven binding — e.g. a React element with `href={userControlledString}` — compiles down to a plain `setAttribute` call. That is a Tier 1, graph-mutation operation: no parse occurs, nothing in §3.2's pipeline ever sees it, because there is no string being turned into a tree. If `userControlledString` is a `javascript:` or `data:` scheme value (or a style value containing `expression()`/behavior injection), it is still exploitable when the element is activated — entirely independent of any HTML-parsing question, and therefore untouched by a design built entirely around "detect a parse, purify the parse."

This is not a hypothetical edge case: because reconciler frameworks default developers onto the safe, non-parsing path for ordinary content, attribute/URL injection through bindings that were never expected to need sanitization is, in practice, at least as common a real-world CSR vulnerability class as raw-HTML injection — arguably more common today, precisely because the raw-HTML escape hatches are visible and rare in most codebases, while ordinary attribute bindings are everywhere.

Recommended second lane

A lightweight check, separate from the seven-stage pipeline, applied at Tier 1 but only for a small, fixed set of attribute names where a value can change program behavior:

- `href`, `src`, `action`, `formaction`, `xlink:href` — scheme allowlist (reject `javascript:`, `data:` where not explicitly needed, `vbscript:`).
- `style` — reject `expression()`, `behavior:`, and `url()` values pointing at disallowed schemes.
- Any attribute name beginning with `on` — reject outright at Tier 1, regardless of source; there is no legitimate reconciler-generated binding that should produce one.

This lane does not invoke Stages 2–7 — there is no tree to stage, purify, or structurally verify, only a scalar value to validate against a scheme allowlist before the existing Tier 1 call proceeds.

3.5 Reconciliation Detail: Updates to Already-Live Nodes

Reconciliation is a diff process. A meaningful share of the DOM operations issued during a commit are updates to nodes already attached to the live document — not fresh, off-document insertions. This is worth stating explicitly for the diagram, because the staging-area framing in §2.2 could otherwise be read as applying only to brand-new content.

It does not require a design change: Stage 2 redirects the raw string to the detached template regardless of whether the eventual target element is already live or not yet attached. The already-live target is not touched until Stage 7, and Stage 7 is a pure graph operation in either case — moving purified, verified children into an existing live parent is exactly as parse-free as moving them into a brand-new one. The diagram should show the target node's attachment state as irrelevant to Stages 1–6, and only relevant at Stage 7 in the trivial sense of "which parent do the verified children get moved into."

3.6 Trusted Types: Relationship, Not Competition

Trusted Types is not a sanitizer and not a staging architecture — it is a type gate enforced at the sink boundary. Once a page sets the CSP directive `require-trusted-types-for 'script'`, the browser itself throws a `TypeError` if a raw string reaches `innerHTML`, `outerHTML`, `document.write`, and the other DOM-XSS sinks; the only value that satisfies the sink afterward is a `TrustedHTML` (or `TrustedScript` / `TrustedScriptURL`) object produced by a registered policy function. The specification says nothing about what that policy function does internally.

Framed this way, Stages 2–6 of §3.2 are exactly the kind of logic a Trusted Types policy function would contain — the two are not alternatives, one is the enforcement perimeter and the other is the content pipeline that perimeter can be configured to call into.

Where they genuinely differ

Property	PseudoDOM wrapper layer (§2–§3.3)	Trusted Types, CSP-enforced
Bypass via early-captured native reference	Possible	Not possible — there is no unprotected native path left once enforced
Coverage across iframe realms	Requires separate injection per realm	Requires the CSP header per realm/document, same underlying limitation
Requires a server-side deployment step	No	Yes, for the enforcing guarantee specifically
Prescribes staging / provenance / structural verification	Yes — this is the whole architecture	No — left entirely to the policy function
Prescribes operation batching	Yes	No

One caveat that keeps this comparison honest: `trustedTypes.createPolicy()` without the enforcing CSP directive is just as opt-in and just as bypassable as a pure wrapper system — the non-bypassable guarantee is a property of the CSP enforcement mode specifically, not an inherent property of the Trusted Types API on its own. "Client-side only, no server changes required" is a real, fair advantage of the PseudoDOM design as proposed in §2.6 — it comes paired with the same bypass exposure Trusted Types has in its own non-enforced mode.

Recommendation

Expose Stages 2–6 as a single, named policy function. Client-side-only deployments get the full pipeline unconditionally, matching the original goal in §2.6. Deployments that can also set the CSP header register that same function with `trustedTypes.createPolicy`, so any code path that bypasses the framework/wrapper layer entirely — which §3.3 establishes is a real residual risk for a purely opt-in system — still cannot reach a raw sink, without maintaining two separate implementations of the sanitization logic.

3.7 Native Platform Alternative Worth Tracking: the Sanitizer API

`Element.prototype.setHTML()` and the accompanying Sanitizer configuration object are a standards-track, browser-native primitive for "parse a string, sanitize the result against a config, attach it" as a single built-in operation. Availability varies by browser and should be checked against current support before being relied on for a production baseline.

It does not provide staging, batching, event-handler provenance tracking, or the structural-fidelity check in §3.2's Stage 5 — so it does not replace this architecture. It is relevant as a possible delegate for the raw parse-and-strip portion of Stage 4 where the browser supports it, with `DOMPurify` (or the manual in-place walk) retained as the fallback where it is not, and the custom layer (Stages 1–3, 5–7) unchanged either way.

4. Consolidated Comparison Table

Mechanism	Enforced by	Bypass-resistant?	Server config?	Tree-shape (§1.2)	Attr/URL (§3.4)	Batching	Handler provenance
Raw DOM, unprotected	Nothing	N/A	No	No	No	No	No
PseudoDOM — Part One	App wrapper	No	No	Yes	No	Yes	Partial
PseudoDOM — Part Two	Framework render path	Within framework	No	Yes	Yes	Yes	Yes
Trusted Types, CSP-enforced	Browser engine	Yes	Yes	Policy-dependent	Policy-dependent	No	No
Trusted Types, no CSP	Advisory only	No	No	Policy-dependent	Policy-dependent	No	No
Native Sanitizer API	Browser engine	N/A	No	Per-call	No	No	No
DOMPurify alone	App, manual	No	No	If not re-parsed	Partial	No	No

5. Threat Coverage Matrix

Threat	Covered by Part One?	Covered by Part Two?	Residual risk / notes
Foster-parenting-class tree relocation inside a parsed string	Yes — staged, parsed in isolation	Yes, unchanged from Part One	None once Stage 2 redirect is in place
Adoption-agency-class misnesting recovery inside a parsed string	Yes, same mechanism as above	Yes, unchanged	None
mXSS via sanitize-then-reparse drift	Partial — depends on DOMPurify mode used	Yes — §3.2 Stage 4 mandates in-place purification, no round trip	Closed specifically by the Stage 4 refinement
Inline event-handler injection via a parsed string	Partial — tracked, but not split by provenance	Yes — §3.2 Stage 3 splits string-sourced vs app-sourced	Part One's undifferentiated whitelist risked over- or under-trusting
Fusion-across-boundaries bypass (content split across multiple calls)	No — batching order not specified	Yes — §3.2 Stage 6 mandates purify-after-aggregate	Closed specifically by the Stage 6 ordering rule
Attribute/URL scheme injection (javascript:, data:, style expressions)	No — out of scope for a string-sink-only design	Yes — §3.4 second lane	Requires the separate Tier 1 lane; not covered by Stages 1–7 at all
Wrapper bypass via early-captured native reference	No mitigation specified	Reduced — host-config hook removes this for in-framework code; Trusted Types pairing (§3.6) closes it fully when CSP is available	Residual for code outside the framework and without CSP
Iframe realm not covered by interception	No mitigation specified	Same residual as above	Requires per-realm injection or CSP per document
Already-live node updates during reconciliation	Ambiguous in the original framing	Explicitly addressed — §3.5 confirms no design change needed	Clarification only, not a new mechanism

6. Data Flow Specification for the Architecture Diagram

A direct translation of the design above into nodes and edges, for building the formal diagram.

6.1 Entry Points (top of diagram)

- Application / framework code issuing a DOM IDL call
- Tier classifier — the single decision point every call passes through first

6.2 Tier 1 Path (left branch)

1. Graph-mutation call received
2. Attribute-name check: is this one of href / src / action / formaction / xlink:href / style / on* ?
3. If yes — scheme/value validation lane (§3.4) — pass or reject
4. If no — straight to the operation ledger (shared with Tier 2 at Stage 6) for batching

6.3 Tier 2 Path (right branch, the main pipeline)

5. String-sink call received (§3.1 table)
6. Stage 1 — Sink Interception
7. Stage 2 — Redirect to detached <template>.content (parse happens here)
8. Stage 3 — Pre-Purification Inventory, split into two labeled outputs: string-sourced handler list, app-sourced handler list
9. Stage 4 — In-Place Purification (DOMPurify IN_PLACE / RETURN_DOM_FRAGMENT, or native Sanitizer API per §3.7)
10. Stage 5 — Structural Fidelity Check — pass/fail gate
11. On fail: reject / log / fallback placeholder (developer-configurable policy)
12. On pass: append to the shared Operation Ledger (Stage 6)

6.4 Shared Merge Point

- Operation Ledger — receives both the Tier-1 non-dangerous-attribute stream and the Tier-2 verified-fragment stream
- Flush trigger — scheduled (e.g. next animation frame) or explicit
- Stage 7 — Promotion — Tier 1 operations only, applied to real targets, live or not-yet-attached

6.5 Two Optional Perimeter Integrations (side inputs, not part of the main flow)

- **Host-config hook (§3.3)** — an alternate entry point feeding the same Tier classifier, used for calls originating from a known reconciler's render path instead of from patched global prototypes.

- **Trusted Types policy registration (§3.6)** — Stages 2–6 exposed as one callable policy function, registered with `trustedTypes.createPolicy` when a CSP directive is present, sitting outside the wrapper-bypass risk entirely for calls the browser itself gates.

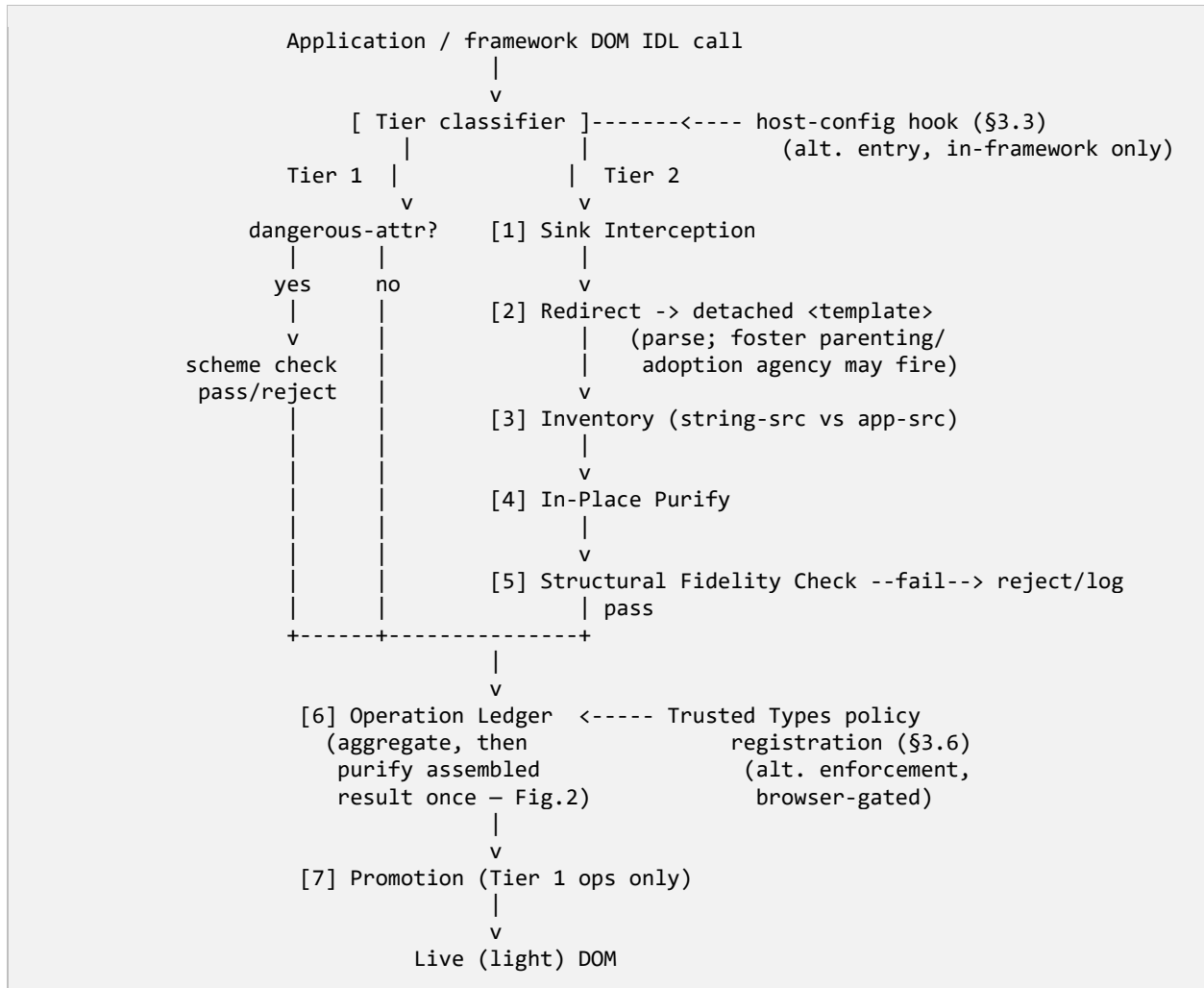


Figure 3. Full system diagram source — both tiers, the shared ledger/promotion path, and the two optional perimeter integrations.

7. Summary and Final Recommendation

Part One is sound as an application-level pipeline for the raw-HTML injection class specifically: staging, tracking, and sanitizing before attachment is the correct shape for the risk that string-sink calls create, and nothing in Part Two contradicts that core design.

Part Two closes three gaps that only become visible once the deployment target is stated precisely as CSR reconciler frameworks:

13. Interception robustness — hooking the framework’s host config / custom renderer, rather than relying solely on global prototype patching, removes the early-reference and operation-inference weaknesses for anything the framework itself renders.
14. A coverage class Part One does not touch at all — attribute and URL-value injection through ordinary, non-parsing bindings, which needs its own lightweight, separate lane rather than being folded into the parse-and-purify pipeline.
15. The enforcement-boundary gap — pairing the pipeline with Trusted Types (registered as a policy, when CSP is available) closes the residual bypass risk that any purely opt-in, wrapper-based system has on its own, without giving up the client-side-only deployment Part One was designed around.

7.1 Validation Steps Before Committing to the Diagram as Final

- Benchmark Stages 1–7 at realistic flush volume for the target application’s highest-frequency mutation site — four tree walks per flush (Stage 2 parse, Stage 3 inventory, Stage 4 purify, Stage 5 verify) is not free.
- Decide whether sanitization policy needs to vary per staged region (§3.2 assumes one policy per flush; per-region policy needs a design note in the Stage 4/5 boxes).
- Confirm the Tier 2 enumeration in Appendix A stays current as new browser APIs ship — the risk model depends on that list being exhaustive, not the pipeline logic.

Appendix A — Tier 2 (String-Sink) Surfaces to Intercept

API	Notes
Element.prototype.innerHTML (setter)	Accessor property — requires descriptor override, not a method reassignment
Element.prototype.outerHTML (setter)	Same as above; also replaces the element itself in its parent
Element.prototype.insertAdjacentHTML	Takes a position argument in addition to the string
document.write / document.writeln	Only meaningful during the initial parse in practice; still worth gating
Range.prototype.createContextualFragment	Easy to miss — a full fragment parse, same algorithm family as innerHTML
DOMParser (constructor) + parseFromString	Wrap the class if the app or its dependencies use it directly
Element.prototype.setHTML (Sanitizer API, where supported)	Native parse+sanitize primitive — see §3.7 for how it fits the pipeline

Appendix B — Dangerous Attribute Names for the Tier 1 Lane (§3.4)

Attribute	Validation
href, xlink:href	Scheme allowlist (http:, https:, mailto:, relative paths); reject javascript:
src	Scheme allowlist; context-dependent (script src is a separate, higher-risk case)
action, formaction	Scheme allowlist; same-origin policy where applicable
style	Reject expression(), behavior:, and url() pointing at disallowed schemes
on* (any attribute name with this prefix)	Reject unconditionally at Tier 1, regardless of value or source